

Assignment 2 - Advanced Programming 2026

Anton Bang Lejbølle & Jonnas Dennis Hansen

September 2026

1 Introduction

For this assignment we do not have any general concerns or have found any ambiguities in the problem text. We are confident that our solution is of high quality as it is functional and passes all the test cases we have written for it. There might still be some edge cases that we have not considered, but we believe that our solution is robust and handles the majority of scenarios effectively.

Our printing function and key-value store functions are functional, and we have written various test cases to cover their functionality. The test cases ensures that the printing function correctly formats and outputs the expected results, and that both the printing and the key-value store functions have the correct order. Most notably we have ensured to test both printing and key-value store functions, when local environments are used in forloops and apply expressions, such that the prints and key-value store functions globally.

Our type checker is functional, and we have written a comprehensive set of test cases to cover its functionality. The test cases include various positive and negative scenarios, which all pass. Since the type checker has implemented the environment almost identically to how it was implemented in the evaluator, we are confident that it is functional. The only differences in the implementation is that the environment does not have a state as this is not needed, and that we check the lambda expressions even if they are not called in an apply.

2 Answers to questions

1. What is the asymptotic complexity of your implementation of Print? Can this be improved?

Our implementation of Print is $O(n + m)$ where n is the amount prints already done and m is the length of the string s .

This is because the printed string is added to the end of the list of already printed strings. In haskell this is done by the $++$ operator which is $O(n)$ where n is the length of the first list.

Similarly, the string is generated by adding `" : "` and $p(v)$ (the textual representation of v) to the end of s using the $++$ operator. As strings are just lists

of chars in haskell, the $++$ operator is $O(n)$ where n is the length of the first list.

This could be improved by adding new strings to the front of the list instead of to the end, and then the complexity would just be $O(m)$, and only depend on the input of the Print statement. To return the correct order of the strings, such that the first string to be printed comes first in the list, we could just add that before returning the list of strings, we reverse the order. Reversing the order of the list will take $O(n)$ where n is the length of list, but as it only needs to be done once, it is therefore $O(1)$ for each print.

2. In your implementation, if a program keeps updating the same key with KvPut, does the memory usage of the program remain constant or does it increase?

The memory usage increases since the new key value pair is just added to the list of key value pairs.

To avoid this we could either implement KvPut such that it tries to find the key, and just updates the value if it is already there, but this would increase the time of KvPut from $O(1)$ to $O(n)$ where n is the length of the Key-value list.

The best way would be to implement the Key-value store as a hash table, as this would keep the time of KvPut and KvGet to $O(1)$, and keep the memory usage of the program constant even if it keeps updating the same key.

3. In TryCatch e1 e2, if e1 fails after performing some effects, are those effects visible in e2, and which of your tests demonstrate this? If you wanted different behaviour (either making the effects visible or invisible), what would you need to change in your implementation? Show exactly what the new code would look like.

Currently if $e1$ fails after performing some effects, those effects are still visible in $e2$.

For printing this is demonstrated in the test called "TryCatch preserves printed output" and for the key-value store it is demonstrated in the test called "Try putkey and fail keeps key in catch".

To change this, we would need to change which state we use when evaluating $e2$. Instead of using the state given from the evaluation of $e1$, we would need to "rollback" and use the original state.

The new code for catch would look like this, where the only change is the case where $e1$ fails now returns " $m2 (env, st)$ " instead of " $m2 (env, st1)$ "

Listing 1: New code for catch

```
catch :: EvalM a -> EvalM a -> EvalM a
catch (EvalM m1) (EvalM m2) = EvalM $ \(env, st) ->
  case m1 (env, st) of
```

```
(st1, Left _) -> m2 (env,st)
(st1, Right x) -> (st1, Right x)
```

4. Would it be possible to define type class instances instance Monad m => Semigroup (m ()) where ... instance Monad m => Monoid (m ()) where ... and what might the definitions for mempty and <> look like? If so, argue based on the monad laws that your instances satisfy the monoid laws.

Yes, it is possible to define both instances using the Monad constraint. We can define the `Semigroup` instance as

```
instance Monad m => Semigroup (m ()) where
  (<>) = (>>)
```

and the `Monoid` instance as

```
instance Monad m => Monoid (m ()) where
  mempty = pure ()
```

The operator `(>>)` evaluates the first monadic action, ignores its result, and then evaluates the second action. Therefore, defining

$$x <> y = x >> y$$

means that the two actions are performed in sequence.

The `Semigroup` law requires associativity:

$$(x <> y) <> z = x <> (y <> z).$$

With the above definition, this becomes

$$(x >> y) >> z = x >> (y >> z),$$

which follows from the associativity law of the `Monad`.

For the `Monoid` instance, `mempty` is defined as

$$\text{mempty} = \text{pure } ().$$

This acts as an identity element. The left identity law becomes

$$\text{mempty} <> x = \text{pure } () >> x = x,$$

which follows from the left identity monad law. Similarly, the right identity law becomes

$$x <> \text{mempty} = x >> \text{pure } () = x,$$

which follows from the right identity monad law.

Therefore, the `Semigroup` and `Monoid` instances satisfy the required laws, since the `Semigroup` associativity and `Monoid` identity laws follow from the `Monad` laws.